데이터 분석 실습 가이드

미세먼지와 날씨의 관계로 배우는 “분석” 단계

> *이 자료는 특정 주차용으로 확정된 자료가 아니라, 커리큘럼 파이프라인(수집→정제→팩트체크→기사화→시각화)에 빠져 있던 “분석” 단계를 채우기 위한 독립 실습 예시입니다. 실제 주차 배치는 추후 별도로 결정합니다.*

1. 이 실습의 목적

지금까지 정리한 5 단계 파이프라인(수집 → 정제 → 팩트체크 → 기사화 → 시각화)에는 “분석”이라는 단계가 명시적으로 없었습니다. 학생들이 데이터를 모으고 정리하는 법은 배우지만, 그 데이터로 가설을 세우고 검증하는 과정은 따로 다루지 않았다는 뜻입니다. 이 실습은 그 공백을 채우기 위한 구체적인 사례입니다.

“미세먼지 피해를 예방·대비하려면 어떤 데이터를 봐야 할까”라는 질문에서 출발해, 공공데이터 두 종(에어코리아 대기오염정보, 기상청 관측자료)을 API 로 직접 수집하고, “바람이 강하거나 비가 오면 미세먼지가 낮아질 것이다” 라는 가설을 실제 데이터로 검증합니다.

이 실습에서 얻어가는 핵심 교훈

- 상관계수(평균적인 직선 관계) 하나만 보고 “관계없음”이라 단정하면 안 된다 — 구간별 최댓값·분포까지 봐야 진짜 패턴이 드러난다.
- 공공데이터 API 는 대량 요청 시 타임아웃이 흔하다 — 페이지네이션 + 재시도 로직이 실무 기본기다.
- 결측치는 무조건 삭제하면 안 된다 — 삭제가 오히려 분석을 왜곡시킬 수 있다.
- 원본 API 가 주는 수십 개 컬럼 중 분석 목적에 맞는 것만 추려 쓰는 판단력이 필요하다.

2. 실습 개요

항목	내용
대상 지역	수원시 신풍동 (에어코리아 측정소) / ASOS 지점 119 번(수원)
기간	2026 년 8 월 1 일 ~ 31 일 (744 시간)
사용 데이터	① 에어코리아 대기오염정보(측정소별 실시간 측정정보) ② 기상청 지상(종관,ASOS) 시간자료
도구	Python(pandas, requests, matplotlib) · Google Colab
노트북 구성	① 수집 노트북(API 호출·원본 저장) ② 정제·분석 노트북(업로드→정제→병합→분석)

> *노트북을 수집용과 정제·분석용으로 분리한 이유: 한 번 수집한 원본을 CSV 로 남겨두면, 다른 지역·다른 기간으로 바꿀 때도 정제·분석 노트북을 그대로 재사용할 수 있습니다. 3 주차부터 써온 “코랩 표준 3 셀 구성”의 확장판이라고 보면 됩니다.*

3. 1 단계 — 공공데이터포털 API 키 발급

두 데이터 모두 공공데이터포털(data.go.kr)의 표준 절차로 키를 발급받습니다: 회원가입 → 데이터셋 검색 → 활용신청 → (대부분 즉시 승인) → 마이페이지에서 서비스키 확인.

데이터	검색어	필요한 오퍼레이션(상세기능)
에어코리아	한국환경공단_에어코리아_대기오염정보	측정소별 실시간 측정정보 조회(getMsrstnAcctoRltmMesureDnsty), 시도별 실시간 측정정보 조회(getCtprvnRltmMesureDnsty)
기상청	기상청_지상(종관,ASOS) 시간자료 조회서비스	getWthrDataList

> *실무 팁: 파이썬 requests 로 호출할 때는 Encoding 키가 아니라 Decoding 키를 써야 합니다. Encoding 키를 쓰면 URL 인코딩이 중복되어 오류가 납니다.*

4. 2 단계 — 측정소·관측지점 확인

에어코리아: 정확한 측정소명 찾기

에어코리아는 “수원” 같은 시 단위 이름이 아니라, 동 단위의 정확한 측정소명이 필요합니다(예: 신풍동, 영통동). 이름을 추측하는 대신, “시도별 실시간 측정정보 조회”로 경기도 전체 측정소 목록을 받아 실제 존재하는 이름을 확인하는 것이 정확합니다.

```
import requests, pandas as pd

SERVICE_KEY = "발급받은_디코딩_인증키"
url = "https://apis.data.go.kr/B552584/ArpltnInforInqireSvc/getCtprvnRltmMesureDnsty"
params = {
    "serviceKey": SERVICE_KEY, "returnType": "json",
    "sidoName": "경기", "ver": "1.3", "numOfRows": 200, "pageNo": 1,
}
res = requests.get(url, params=params)
df = pd.DataFrame(res.json()["response"]["body"]["items"])
print(sorted(df["stationName"].tolist()))
```

이 방법으로 수원의 측정소 5 곳(신풍동·광교동·영통동·천천동·고색동)을 확인했고, 이 중 데이터가 안정적으로 들어오는 신풍동을 최종 선택했습니다.

기상청 ASOS: 지점번호 확인

ASOS 는 지역명이 아니라 지점번호(stnIds)로 조회합니다. 수원은 119 번으로 알려져 있는데, 실제로 맞는지는 테스트 호출 응답의 stnNm(지점명) 필드로 바로 검증할 수 있습니다 — 지점번호를 외우기보다 이렇게 확인하는 습관이 중요합니다.

5. 3 단계 — 데이터 수집

3 개월치(2,200 여 건)를 numOfRows 를 크게 잡아 한 번에 요청했더니 504 SERVICETIMEOUT_ERROR 가 간헐적으로 발생했습니다. 공공데이터 API 에서 흔한 문제로, 페이지당 요청 건수를 줄이고(300~500 건), 실패하면 자동 재시도하는 방식으로 해결했습니다.

문제 상황	원인	해결
504 SERVICETIMEOUT_ERROR (간헐적)	numOfRows 를 너무 크게(3000+) 요청해 서버 처리시간 초과	numOfRows 를 300~500 으로 줄이고 pageNo 로 나눠서 요청
requests.exceptions.ReadTimeout 예외로 코드 전체가 죽음	res.json() 파싱 오류만 잡고, requests.get() 자체의 타임아웃은 못 잡음	requests.get() 호출도 try/except 로 감싸서 재시도 대상에 포함
최종확정 측정자료(	그건 QA 검증을 거친 “확정치”	실시간 측정정보 API 와 확정자료

문제 상황	원인	해결
에어코리아 웹페이지)가 4월까지만 보임	라서 원래 몇 달 지연됨 (우리가 쓰는 API 는 실시간/잠정치라 무관)	페이지는 다른 데이터라는 점을 구분

수집 함수 (재시도 + 페이지네이션)

```
import requests, pandas as pd, time

SERVICE_KEY = "발급받은_디코딩_인증키"
url = "https://apis.data.go.kr/B552584/ArpltnInforInqireSvc/getMsrstnAcctoRltmMesureDnsty"

def fetch_air_page(page_no, station="신풍동", num_rows=500, max_retry=4):
    params = {
        "serviceKey": SERVICE_KEY, "returnType": "json",
        "stationName": station, "dataTerm": "3MONTH", "ver": "1.3",
        "numOfRows": num_rows, "pageNo": page_no,
    }
    for attempt in range(1, max_retry + 1):
        try:
            res = requests.get(url, params=params, timeout=30)
            data = res.json()
        except (requests.exceptions.RequestException, ValueError):
            time.sleep(3); continue
        if "response" in data:
            return data["response"]["body"]
        time.sleep(3)
    return None

first = fetch_air_page(1)
total_pages = -(-first["totalCount"] // 500)
items = list(first["items"])
for p in range(2, total_pages + 1):
    body = fetch_air_page(p)
    if body: items.extend(body["items"])
    time.sleep(0.5)

df_air_raw = pd.DataFrame(items)
```

기상청 ASOS 수집 (응답 구조가 다름: items.item)

```
url = "https://apis.data.go.kr/1360000/AsosHourlyInfoService/getWthrDataList"

def fetch_weather_page(page_no, stn="119", start="20260801", end="20260831",
                       num_rows=300, max_retry=4):
    params = {
        "serviceKey": SERVICE_KEY, "dataCd": "ASOS", "dateCd": "HR",
        "stnIds": stn, "startDt": start, "startHh": "00",
        "endDt": end, "endHh": "23",
        "numOfRows": num_rows, "pageNo": page_no, "dataType": "JSON",
    }
    for attempt in range(1, max_retry + 1):
        try:
            res = requests.get(url, params=params, timeout=30)
            data = res.json()
        except (requests.exceptions.RequestException, ValueError):
            time.sleep(3); continue
        if "response" in data and data["response"]["header"]["resultCode"] == "00":
            return data["response"]["body"]
        time.sleep(3)
    return None

first = fetch_weather_page(1)
total_pages = -(-first["totalCount"] // 300)
items = list(first["items"]["item"])   # 주의: items 밑에 item이 한 번 더 있음
```

```
for p in range(2, total_pages + 1):
    body = fetch_weather_page(p)
    if body: items.extend(body["items"]["item"])
    time.sleep(0.5)

df_weather_raw = pd.DataFrame(items)
```

6. 4 단계 — 정제

왜 이 컬럼만 남기는가

에어코리아 원본은 26 개, ASOS 원본은 38 개 컬럼을 줍니다. 분석 목적(미세먼지-날씨 관계)에 맞는 것만 추립니다.

데이터	남기는 컬럼	남기는 이유
에어코리아	dataTime, pm10Value, pm25Value	시간 기준 병합 키 + 이번 분석의 핵심 대상(미세먼지 농도)
	(제외) so2Value, coValue, no2Value, o3Value/o3Grade 등	아황산가스·일산화탄소·이산화질소·오존 등 다른 오염물질 수치 — 이번 주제와 무관
	(제외) mangName, *Flag, *Qcflg	측정망 구분·품질검증 플래그 — 분석에 쓰지 않음
ASOS	tm, ta, rn, ws, hm	시간 기준 병합 키 + 가설 검증에 필요한 기온·강수량·풍속·습도
	(제외) ps, pa, ss, vs, 지중온도 등	해면기압·일조·시정·지중온도 등 — 이번 가설과 직접 관련 없음

결측치: 삭제 대신 보간을 선택한 이유

미세먼지 713 행 중 PM10 13 개, PM2.5 16 개가 결측이었습니다. 무조건 삭제(dropna)하지 않고 먼저 결측이 얼마나 연속되는지부터 확인했습니다 — 통신장애 같은 결측이 하필 농도가 극단적으로 높을 때 더 자주 발생하는 경우가 있어서, 무작정 지우면 “가장 위험했던 순간”이 통계에서 빠지는 편향이 생길 수 있기 때문입니다.

```
df_air = df_air_raw[["dataTime", "pm10Value", "pm25Value"]].copy()
df_air["dataTime"] = pd.to_datetime(df_air["dataTime"], errors="coerce")
df_air["pm10Value"] = pd.to_numeric(df_air["pm10Value"], errors="coerce")
df_air["pm25Value"] = pd.to_numeric(df_air["pm25Value"], errors="coerce")

df_weather = df_weather_raw[["tm", "ta", "rn", "ws", "hm"]].copy()
df_weather["tm"] = pd.to_datetime(df_weather["tm"], errors="coerce")
for col in ["ta", "ws", "hm"]:
    df_weather[col] = pd.to_numeric(df_weather[col], errors="coerce")
# 강수량 빈칸 = 비 안 옴 → 0 (기온·풍속·습도는 결측이면 그대로 NaN 유지)
df_weather["rn"] = pd.to_numeric(df_weather["rn"], errors="coerce").fillna(0)

# 결측이 몇 시간이나 연속되는지 먼저 확인 (길면 보간 대신 다른 처리 필요)
na_mask = df_air["pm10Value"].isna()
na_groups = (na_mask != na_mask.shift()).cumsum()[na_mask]
print(sorted(na_mask.groupby(na_groups).sum().tolist(), reverse=True))
# 결과: [3, 2, 2, 1, 1, 1, 1, 1, 1] → 최대 3시간, 보간해도 안전한 수준
```

7. 5 단계 — 병합 + 보간

```
merged = pd.merge(
    df_air, df_weather, left_on="dataTime", right_on="tm", how="inner"
).drop(columns=["tm"])
merged = merged.sort_values("dataTime").reset_index(drop=True)

# 짧게 흩어진 결측이므로 선형보간 (양쪽 방향)
merged["pm10Value"] = merged["pm10Value"].interpolate(method="linear",
limit_direction="both")
merged["pm25Value"] = merged["pm25Value"].interpolate(method="linear",
limit_direction="both")
# → 병합 713행, 결측 0으로 정리 완료
```

8. 6 단계 — 분석: 상관계수의 함정

1 차 시도: 상관계수

```
corr_ws = merged["pm10Value"].corr(merged["ws"])
corr_rn = merged["pm10Value"].corr(merged["rn"])
# PM10-풍속 상관계수: -0.040
# PM10-강수량 상관계수: -0.063
```

두 값 모두 0 에 가깝습니다. 여기서 “바람·비는 미세먼지와 관계없다”고 결론짓기 쉽지만, 산점도를 직접 그려보면 다른 이야기가 보입니다.

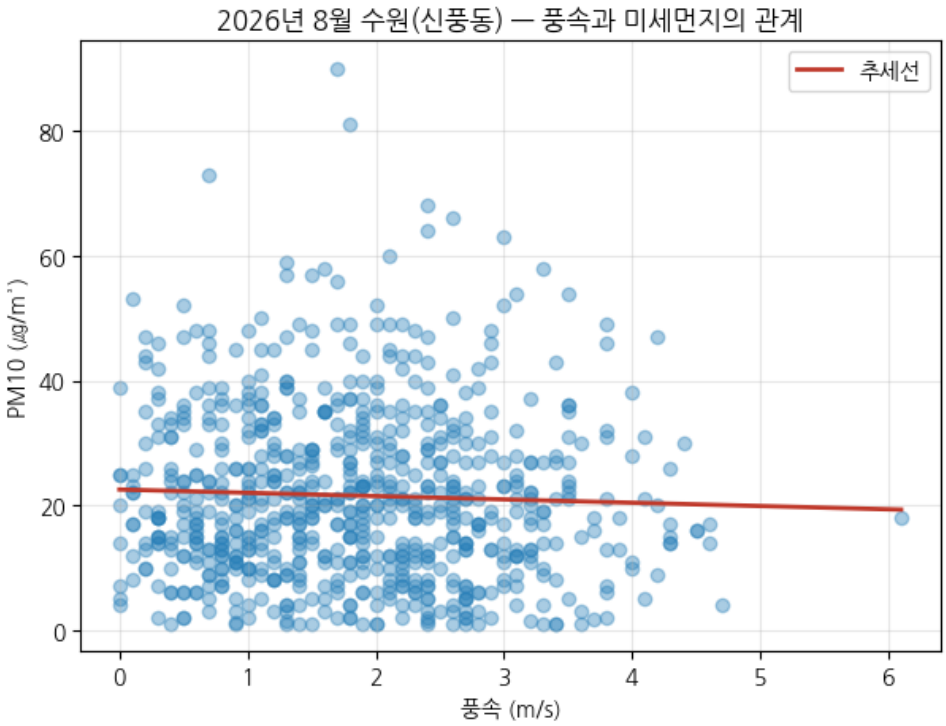


그림 1. 풍속과 PM10 의 산점도 — 추세선은 거의 평평하지만, 높은 PM10 값(60 이상)은 전부 풍속 1~2.5m/s 구간에 몰려 있다

2 차 시도: 구간별 최댓값·상위 90%

“평균”이 아니라 “극값”을 구간별로 비교하면 패턴이 뚜렷해집니다. 풍속이 세질수록 PM10 의 평균은 비슷하지만, 최댓값과 상위 90% 값은 꾸준히 낮아집니다 — 바람이 강하면 “아주 나쁨” 상황 자체가 잘 생기지 않는다는 뜻입니다.

풍속 구간(m/s)	평균	최댓값	상위 90%
0-1	21.1	73.0	38.0
1-2	23.2	90.0	40.0
2-3	21.1	68.0	42.0
3-4	20.0	58.0	36.1
4+	18.6	47.0	30.3

강수 구간	표본 수	평균	최댓값	상위 90%
0 (비 없음)	648	21.9	73.0	40.0
0~1mm	36	20.4	90.0	37.5
1~5mm	22	14.0	42.0	33.5
5mm+	7	16.6	31.0	26.8

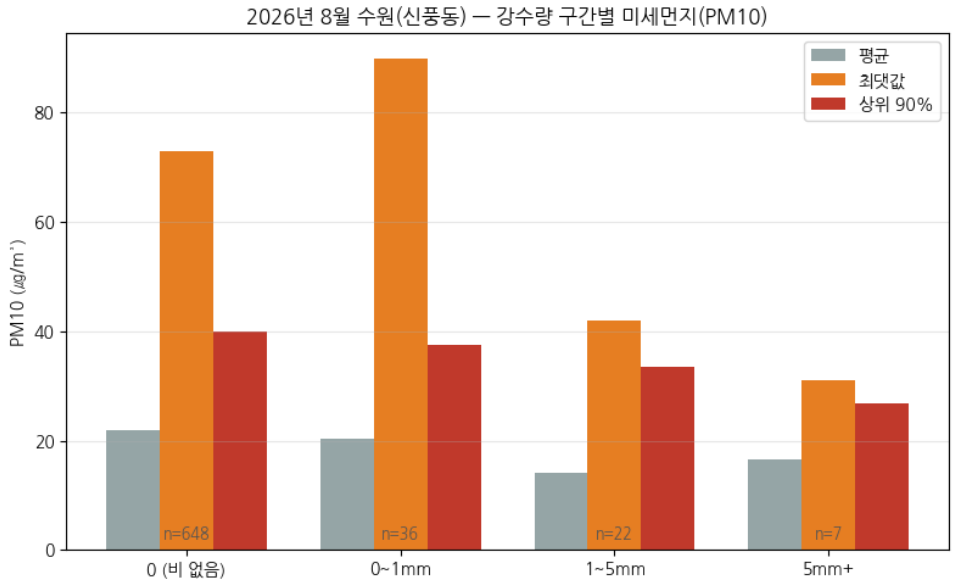


그림 2. 강수량 구간별 PM10 비교 — 상위 90%(빨간 막대)는 비가 많아질수록 꾸준히 낮아진다. 0~1mm 구간의 최댓값(주황)이 튀는 것은 표본이 36 개뿐이라 생긴 예외적 노이즈

> *주의: 1~5mm(22 개), 5mm+(7 개) 구간은 표본이 적어 신뢰도가 낮습니다. "표본 수가 적을 때는 최댓값 같은 극값 통계를 그대로 믿으면 안 된다"는 것도 학생들에게 함께 가르칠 만한 포인트입니다.*

9. 결론 및 예방·대비 시사점

평균만 보면 "풍속·강수와 미세먼지는 관계가 없다"로 오판하기 쉽지만, 극값(최댓값·상위 90%)을 보면 "바람이 약하고 비가 없는 날은, 평소엔 괜찮아 보여도 미세먼지가 갑자기 심해질 가능성 자체가 더 크다"는 실용적인 결론을 얻을 수 있습니다. 이는 "오늘 평균 농도"보다 "오늘 같은 기상 조건에서 최악의 경우 얼마나 나빠질 수 있는가"가 예방·대비에는 더 중요한 정보일 수 있다는 뜻입니다.

10. 학생 실습으로 전환할 때 고려할 점

- 지역·기간을 학생마다 다르게 배정 가능 — "시도별 실시간 측정정보 조회"로 받은 측정소 목록에서 각자 다른 동네를 고르게 하면 결과 비교가 흥미로워짐.
- 표본 수가 적은 구간을 해석할 때 주의하는 법 — 이번 강수 1~5mm/5mm+ 구간(n=22, n=7) 사례를 그대로 예시로 쓸 수 있음.

- 결측치 처리(삭제 vs 보간)를 학생이 직접 판단해보게 하는 것도 좋은 토론거리 — “왜 삭제가 위험할 수 있는가”를 스스로 설명해보게 하기.
- 상관계수 → 산점도 → 구간별 통계로 이어지는 “한 가지 지표만 믿지 않는” 분석 태도 자체가 이 실습의 핵심 메시지.